



Beni-Suef University

Faculty of Engineering - Electrical Engineering Department



Graduation Project

Release BSU Engineering Portal: A Comprehensive Academic Management System

Academic Supervisor

Associate Prof. Dr. Fathy Mostafa

Project Metadata

Academic Year: **2025-2026**

Presentation Date: **June 2026**

Project Members

Mahmoud Hussein Mahmoud

Mohamed Gamal Shehata

Mohamed Essam Sayed



Table of Contents

1 Abstract

3 Introduction and Objectives

5 Methodology and Design

7 Testing and Performance

9 Conclusions and Practical Implications

2 Acknowledgements

4 Theoretical Background

6 Cloud Deployment Architecture

8 Results and Discussion

10 Future Work

Abstract



The Operational Challenge

Fragmented, Excel-based workflows caused severe data inconsistencies, lacked audit trails, delayed grade processing, and hindered self-service access for over 4,000 enrolled students.



The Architectural Solution

Deployed a modern, decoupled web architecture featuring a Django 5.0 REST API (providing secure, stateless endpoints with CSRF protection) and a responsive React 19 frontend utilizing Material-UI.



Infrastructure and Data Persistence

Utilizes MySQL 8.0 to ensure strict ACID compliance for sensitive grade and attendance transactions, alongside Redis for caching and Docker for portable containerization.

Abstract



Core System Capabilities

Features hierarchical academic modeling (departments → specializations → levels → students), transaction-safe batch CSV/Excel imports for enrollments and grades using Pandas, and an interactive multimodal quiz engine.



Security and Impact

Enforces the principle of least privilege across eight distinct roles (Admin, Dean, HOD, Doctor, Student, etc.) via custom Role-Based Access Control (RBAC) and comprehensive audit logging, completely eliminating institutional data silos.

Acknowledgements



Academic Supervision

We extend our sincere gratitude to Associate Prof. Dr. Fathy Mostafa for his invaluable guidance, patience, and unwavering support throughout this project.



Institutional Support

We thank the faculty members and staff of the Faculty of Engineering, Beni-Suef University, whose insights were vital in understanding the core requirements and operational challenges of the institution.



Family Support

Our deepest gratitude goes to our families and peers for their endless support, continuous encouragement, and willingness to beta-test early versions of the portal during late debugging sessions.



External Organizations

We acknowledge the robust open-source communities behind Django, React, Pandas, and Docker for providing the reliable frameworks that form the backbone of this system.

Introduction and Objectives



Project Background

Beni-Suef University historically relied on manual, Excel-based workflows for academic management. This approach created data inconsistencies across departments, lacked audit trails, and scaled poorly with over 4,000 enrolled students.



Problem Statement

The Faculty of Engineering lacks a unified, role-based digital platform for managing core academic operations. This results in severe data fragmentation, operational inefficiencies, critical accountability gaps, and delayed access to essential academic information.



Main Objectives

Deliver a full-stack, role-based academic portal centralizing operations across four key functional pillars: Student Lifecycle Management, Faculty Course Management, Administrative Operations, and a Public Information Portal.

Introduction and Objectives



Project Scope

Five Core Functional Pillars:

- Student Lifecycle Management
- Faculty Course Management
- Administrative Operations
- Alumni Services & Public Information Portal

Deliberate Boundaries and Constraints:

Focused strictly on core administrative reliability. Native mobile applications and real-time chat systems are explicitly excluded.



Expected Deliverables

- **Containerized Web Platform**
Fully containerized deployment architecture featuring a modern decoupled React 19 frontend and a robust Django 5.0 REST API.
- **Role-Based Security**
Secure, fully isolated management dashboards custom-tailored to serve 8 distinct stakeholder roles within the institution.
- **High-Performance Batch Processing**
Transaction-safe processing pipelines capable of instantly handling and updating hundreds of student records.

Theoretical Background



Theoretical Foundations

- **Relational Normalization**
The database schema is strictly normalized to the Third Normal Form (3NF) to eliminate data redundancy and prevent update anomalies, utilizing composite unique constraints.
- **Access Control Theory**
Implements the "Principle of Least Privilege" via a custom Role-Based Access Control (RBAC) matrix that isolates capabilities across 8 distinct stakeholder roles.



Security Framework

- **Cryptographic Protection**
Passwords are cryptographically secured using the PBKDF2-SHA256 hashing algorithm with a 720,000-iteration key derivation function.
- **CSRF & CORS Defense**
The architecture relies on a strict Cross-Site Request Forgery (CSRF) handshake and CORS origin whitelisting rather than JWTs, deliberately chosen for immediate session revocation.

Methodology and Design



Design and Architecture

1. Design Methodology

Adopted an iterative, 10-sprint Agile methodology. This flexible approach allowed the team to deliver features early and dynamically adapt to new faculty requirements mid-development, such as the addition of the multimodal quiz engine.

2. System Architecture Design

Engineered a decoupled, four-tier containerized cloud architecture separating the Presentation, API, Data, and Routing layers.



3. Software and Tools

Frontend

Built with React 19 and Vite, utilizing Material-UI (MUI) to deliver a native Right-to-Left (RTL) Arabic interface.

Backend

Powered by Django 5.x and Django REST Framework (DRF) for stateless API endpoints.

Database and Cache

MySQL 8.0 for strict ACID-compliant persistence, and Redis 7.x for sub-millisecond storage and caching.

Cloud Deployment Architecture



Compute and Network Tier

Live Deployment

The production environment is fully deployed, secured with SSL/TLS encryption, and publicly accessible at <https://bsu-engineering.app/>.

Serverless Compute Tier

Utilizing Azure Container Apps (ACA) to host both the React frontend and Django backend containers, providing HTTP-based auto-scaling.

Infrastructure as Code (IaC)

The entire Azure deployment topology is provisioned dynamically via Terraform, ensuring a deterministic, version-controlled cloud environment.



Data & Storage Tier

Database Tier

Leveraging Azure Database for MySQL (Flexible Server) to ensure a fully managed, highly available relational data tier with strict private endpoint isolation.

Session and Caching Layer

Deploying Redis alongside the application within ACA to handle ultra-fast session storage and sub-millisecond data caching, allowing backend horizontal scaling.

Cloud Object Storage

Integrating Azure Blob Storage to provide centralized, scalable hosting for static assets and user-uploaded media files.

Testing and Performance



Quality Assurance and Security

1. Multi-Layered Quality Assurance

Implemented a comprehensive testing pyramid starting from strict database-level constraints (Layer 1) up to integration testing for grade calculations and atomic transactions (Layer 2).

2. Security and Infrastructure Verifications

Executed continuous container vulnerability scanning using Trivy, achieving zero critical or high-level vulnerabilities, alongside robust RBAC boundary testing (Layer 3).



Performance and Optimization

3. Load Testing and Benchmarking

Conducted simulated heavy-load testing using Locust. The system successfully supported up to 500 concurrent users (Student Grade View) with a 95th percentile response time of just 310ms—well under the 2.0-second requirement.

4. Database and Query Optimization

Addressed N+1 query bottlenecks using Django's `select_related` and `prefetch_related` ORM optimizations, reducing complex nested queries from 41 database hits down to a single SQL JOIN.

Results and Discussion



Operational Efficiency and Latency

1. Massive Efficiency Gains

Automated batch processing reduced the time required to register 500 students from approximately 42 hours down to just 30 seconds—a 99.98% reduction in operational time.

2. Information Latency

Transitioned from a paper-based grade publication pipeline taking 3 to 7 days, to an instant, 24/7 self-service dashboard for students.



Integrity and Accountability

3. Absolute Data Integrity

Replaced multiple disjointed Excel files with a single MySQL source of truth, effectively driving the legacy 1–5% human transcription error rate down to exactly 0%.

4. Institutional Accountability

Upgraded the faculty from a purely trust-based environment to an evidence-based system by implementing immutable audit logging for 15 distinct administrative action types.

Conclusions and Practical Implications



Key Conclusions

1. Achievement of Objectives

The project successfully replaced fragmented, error-prone manual workflows with a centralized, role-based platform, completely digitizing the academic lifecycle for over 4,000 students and eliminating institutional data silos.

2. Technical Contributions

Engineered a highly secure, containerized architecture utilizing a decoupled React SPA and Django REST API. The system implements a strict 7-role RBAC model, 14 distinct security layers, and an immutable 15-type audit logging mechanism.

3. Performance Validation

The system achieved a 100% pass rate across a rigorous 5-layer testing pyramid and successfully maintained sub-second latency (310ms 95th percentile) during simulated loads of 500 concurrent users.



Practical Implications

1. Industrial Applications

Serves as a fully deployable, production-ready Academic ERP system that provides an evidence-based, zero-latency digital environment.

2. Economic Benefits

Drastically reduces operational overhead, evidenced by a 99.98% reduction in batch registration time (from 42 hours to 30 seconds) and the total elimination of human transcription errors.

3. Integration Potential

The decoupled REST API contract and Docker containerization provide an extensible foundation that can easily integrate with future university services.

Future Work



System Improvements

1. Automated Backups

Implement scheduled database backups via mysqldump directly to cloud object storage (S3/GCS).

2. HTTPS Automation

Automate SSL/TLS certificate lifecycle management and renewal using Let's Encrypt.

3. Cumulative GPA Service

Develop a high-performance calculation engine dedicated to cumulative GPA metrics.



Scalability and Topologies

1. Container Orchestration

Evolve from single-server compute to horizontal scaling using Docker Swarm or Kubernetes behind a load balancer.

2. Database Read Replicas

Introduce dedicated MySQL read replicas to isolate heavy analytical reporting from live operational transactions.



Additional Features

1. Mobile Companion App

Build a React Native application providing offline-first capabilities and push notifications.

2. Real-Time Interactions

Integrate Django Channels and Redis to support WebSocket-based chat and live notifications.

3. Expanded LMS Capabilities

Add automated exam proctoring, discussion forums, and virtual classrooms.